

THE TRANSFORMER ARCHITECTURE

A complete, ordered deep dive: tokens → embeddings → positions → encoder → self-attention → residuals → LayerNorm → FFN → decoder → masked self-attention → cross-attention → vocabulary projection → softmax → training → inference, with a full translation dry run.

Running example: “Je suis étudiant” → “I am a student”. This guide is designed to be self-contained: every major component is explained before it is used.

Core idea: the encoder builds contextual representations of the source; the decoder uses target context plus cross-attention to those source representations to generate the target one token at a time.

1. The original Transformer at a glance

The original Transformer is an **encoder-decoder** neural architecture introduced for sequence-to-sequence tasks such as translation. The encoder reads the source sentence. The decoder generates the target sentence.

SOURCE → Tokenize → Embedding + Position → Encoder × 6 → Encoder Memory

TARGET PREFIX → Embedding + Position → Decoder × 6 → Linear → Softmax → Next Token

Original Transformer settings: 6 encoder layers, 6 decoder layers, $d_{\text{model}}=512$, 8 attention heads, $d_k=d_v=64$ per head, and $d_{\text{ff}}=2048$.

2. Step 1 — Tokenization

Text is converted into discrete token IDs. A token can be a whole word, subword, punctuation mark or special symbol. For teaching, imagine: Je→101, suis→205, étudiant→781. The network never directly receives the string “étudiant”; it receives its token ID and then its embedding.

Special target tokens commonly include **BOS** (beginning of sequence) and **EOS** (end of sequence).

3. Step 2 — Token embeddings

Token IDs index a learned embedding matrix E . If vocabulary size is V and $d_{\text{model}}=512$, E has shape $V \times 512$.

$\text{embedding}(\text{id}) = E[\text{id}]$

Every token becomes a 512-dimensional vector. These numbers are learned jointly with the rest of the model.

4. Step 3 — Positional information

Attention does not inherently tell the model that token A came before token B. The original Transformer therefore adds a positional vector to each token embedding.

$X = \text{TokenEmbedding} + \text{PositionalEncoding}$

The original paper uses sinusoidal encodings:

$\text{PE}(\text{pos}, 2i) = \sin(\text{pos} / 10000^{(2i/d_{\text{model}})})$

$\text{PE}(\text{pos}, 2i+1) = \cos(\text{pos} / 10000^{(2i/d_{\text{model}})})$

The important idea is that each position receives a distinct numerical signal. Modern Transformers often use other positional methods, but the classic architecture uses sinusoidal position encodings.

5. Step 4 — Encoder self-attention

Self-attention is the main communication mechanism. In encoder self-attention, **Q, K and V all come from the same encoder sequence**. Every source token can compare itself with every other source token.

5.1 Q, K and V

$Q = XW_Q \quad K = XW_K \quad V = XW_V$

Query: what am I looking for? **Key:** what kind of information do I offer for matching? **Value:** what information should be retrieved? These are learned projections, not manually assigned meanings.

5.2 Similarity scores

$\text{Scores} = QK^T$

For X of shape $n \times 512$ and one head with $d_k=64$: Q, K, V are $n \times 64$. QK^T is $n \times n$. Entry (i, j) is the compatibility between query token i and key token j .

5.3 Scaling

$$\text{ScaledScores} = QK^T / \sqrt{d_k}$$

Scaling keeps dot-product magnitudes controlled as dimensionality grows. In the original model $\sqrt{64}=8$.

5.4 Softmax

$$A = \text{softmax}(QK^T / \sqrt{d_k})$$

Softmax is applied row-wise. Each query gets a distribution of attention weights over all source positions. A large weight means that source position contributes strongly.

5.5 Retrieve information

$$\text{Output} = A V$$

The attention weights are multiplied by the Value matrix. Thus each output token is a weighted mixture of information from the sequence.

Complete formula:

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k})V$$

6. What self-attention actually accomplishes

Consider “The animal didn’t cross the street because it was tired.” The representation of “it” can assign different weights to “animal”, “street”, “tired”, and other tokens. After attention, the representation of “it” incorporates context. This is why Transformer representations are contextual.

Attention is therefore best viewed as **learned, content-dependent communication between token representations**.

7. Step 5 — Multi-Head Attention

One attention mechanism produces one learned interaction pattern. Multi-head attention performs several attention operations in parallel, each with separate learned projections.

$$\text{head}_i = \text{Attention}(XW_{Q_i}, XW_{K_i}, XW_{V_i})$$

$$\text{MHA}(X) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W_O$$

Original Transformer: 8 heads × 64 dimensions = 512 dimensions. The heads are concatenated and projected back into d_{model} dimensions.

Different heads can learn different relationships or representation subspaces. Their roles are learned; they are not hard-coded as “grammar head”, “noun head”, etc.

8. Step 6 — Residual connections

After each major sub-layer, the Transformer adds the sub-layer input back to its output:

$$\text{ResidualOutput} = X + \text{Sublayer}(X)$$

This skip connection provides a direct path for information and gradients and makes deep stacks easier to optimize.

9. Step 7 — Layer Normalization

LayerNorm normalizes the features of an individual representation. For vector x :

$$\mu = \text{mean}(x)$$

$$\sigma^2 = \text{mean}((x - \mu)^2)$$

$$x_{\text{hat}} = (x - \mu) / \sqrt{(\sigma^2 + \epsilon)}$$

$$y = \gamma x_{\text{hat}} + \beta$$

γ and β are learned scale and shift parameters. For a Transformer token with 512 features, the normalization is conceptually performed across those 512 features.

Original Transformer: residual addition followed by LayerNorm (post-norm). Many modern Transformers use pre-norm, where LayerNorm is placed before the sub-layer.

10. BatchNorm vs LayerNorm

	BatchNorm	LayerNorm
Statistics	Across batch-related dimensions	Across features of one representation
Depends on other examples	Yes	No
Common use	CNNs / vision	Transformers
Key intuition	Normalize a feature using batch context	Normalize one token/example using its feature values

For Transformer theory, remember: **LayerNorm is the classic normalization used in Transformer blocks; BatchNorm is not the original Transformer normalization.**

11. Step 8 — Feed-Forward Network

Attention allows tokens to communicate. The feed-forward network then performs a learned nonlinear transformation independently at each position.

$$\text{FFN}(x) = \text{ReLU}(xW_1 + b_1)W_2 + b_2$$

$$512 \rightarrow \text{Linear} \rightarrow 2048 \rightarrow \text{ReLU} \rightarrow \text{Linear} \rightarrow 512$$

The same FFN weights are reused at every sequence position, but each token's vector is processed independently. Therefore:

Attention = token-to-token communication.
FFN = per-token computation/transformation.

12. Complete encoder layer

```
X
↓ Multi-Head Self-Attention
↓ add residual
↓ LayerNorm
↓ Feed-Forward Network
↓ add residual
↓ LayerNorm
↓ Encoder output
```

This block is stacked six times in the original architecture.

13. What comes out of the encoder?

After the final encoder layer, we have a matrix H containing one contextual vector per source position:

$$H = [h_1, h_2, \dots, h_n] \quad \text{shape} = n \times 512$$

H is often called encoder memory. It is **not an English translation**. It is a learned, context-rich representation of the source sentence. The decoder will read this memory using cross-attention.

14. Step 9 — Decoder input

The decoder generates the target autoregressively. Suppose the target is "I am a student". At inference, it begins with:

After generating I, the next decoder input is I. After generating am, it becomes I am, and so on.

15. Step 10 — Masked decoder self-attention

Decoder self-attention is causal. When predicting a token, the decoder must not see future target tokens. For four positions:

Query\Key	1	2	3	4
1	✓	blocked	blocked	blocked
2	✓	✓	blocked	blocked
3	✓	✓	✓	blocked
4	✓	✓	✓	✓

Forbidden positions receive a very large negative score before softmax, making their probabilities effectively zero. This is called causal/look-ahead masking.

$$\text{MaskedScores} = \text{Scores} + \text{Mask}$$

16. Step 11 — Cross-Attention: the bridge from encoder to decoder

This is the mechanism that allows the decoder to access the source. Unlike self-attention, Q and K/V come from different places:

Q ← Decoder

K ← Encoder

V ← Encoder

$$\text{CrossAttention} = \text{softmax}(Q_{\text{dec}} K_{\text{enc}}^T / \sqrt{d_k}) V_{\text{enc}}$$

The decoder Query asks: “**Which source information is relevant to what I am generating now?**” Encoder Keys determine the match; encoder Values provide the information retrieved.

17. Cross-attention shapes

If the decoder has m target positions and the encoder has n source positions:

$$Q_{\text{dec}}: m \times d_k$$

$$K_{\text{enc}}: n \times d_k$$

$$Q_{\text{dec}} K_{\text{enc}}^T: m \times n$$

This m×n matrix is especially intuitive: each row is a decoder position, and each column is a source position. It tells each target position how strongly to read from every source position.

18. Translation: where does it happen?

There is no single hard-coded translation dictionary inside the Transformer. Translation emerges from the learned parameters of the entire network.

The encoder learns to turn source tokens into useful contextual representations. The decoder learns target-language patterns and learns, through cross-attention, how source representations should influence target-token predictions.

Source sentence

→ Encoder contextual memory

→ Decoder target context + cross-attention

- Decoder hidden representation
- Linear vocabulary projection
- Softmax
- Next target token

So the actual translation behavior is distributed across embeddings, attention projections, FFNs and output projections learned during training.

19. Step 12 — Decoder residuals, LayerNorm and FFN

A classic decoder layer contains three sub-layers:

1. Masked Multi-Head Self-Attention
2. Encoder-Decoder Cross-Attention
3. Feed-Forward Network

Residual connections and LayerNorm surround these sub-layers. Conceptually:

Masked Self-Attention → Residual + LayerNorm

Cross-Attention → Residual + LayerNorm

FFN → Residual + LayerNorm

20. Step 13 — Linear vocabulary projection

The final decoder hidden state is still d_{model} -dimensional. A learned linear layer maps it into the target vocabulary space.

$$\text{logits} = h W_{\text{vocab}} + b$$

If the vocabulary has 50,000 tokens, the output has 50,000 logits. Each logit is a score for one possible next token.

21. Step 14 — Softmax

$$P(\text{token}_i) = \exp(z_i) / \sum_j \exp(z_j)$$

Softmax converts logits into probabilities. The selected next token is then appended to the target prefix. Generation repeats until EOS.

Greedy decoding chooses the highest-probability token; beam search keeps multiple candidate sequences; sampling methods introduce controlled stochasticity.

22. FULL DRY RUN — “Je suis étudiant” → “I am a student”

A. Source tokenization

Je → 101 suis → 205 étudiant → 781

B. Embedding + positions

$e_{\text{Je}} + \text{PE}_1$

$e_{\text{suis}} + \text{PE}_2$

$e_{\text{étudiant}} + \text{PE}_3$

Now the encoder has three 512-dimensional vectors containing token and position information.

C. Encoder layer 1

Each head projects the three vectors into Q, K and V. For one head, Q/K/V have shape 3×64 .

$$QK^T \rightarrow 3 \times 3 \text{ score matrix}$$

Each row says where that token wants to look. Scale, softmax, and multiply by V. Repeat for eight heads, concatenate, project, add the residual, and normalize.

Then apply the FFN independently to all three positions: 512→2048→512. Add the residual and normalize again.

D. Encoder layers 2–6

The same overall block is repeated with different learned weights. Each layer can further transform the contextual representations. Finally:

$H = [h_{Je}, h_{suis}, h_{\text{étudiant}}]$ shape 3×512

This H is the encoder memory.

E. Generate “I”

Decoder starts with . Its masked self-attention sees only available target context. Cross-attention creates a decoder Query and compares it against encoder Keys for Je, suis and étudiant. The retrieved encoder Values provide source information.

After decoder processing, the final hidden state goes through Linear + Softmax. Suppose I receives the highest selected probability:

$\rightarrow I$

F. Generate “am”

I

Masked self-attention models the target prefix. Cross-attention again consults H. The decoder now combines source information with the fact that it has already generated “I”. Linear + Softmax favors “am”.

$I \rightarrow am$

G. Generate “a”

I am

The decoder uses target-side context and source-side context again. It predicts “a”.

$I \text{ am} \rightarrow a$

H. Generate “student”

I am a

The decoder's target context indicates what kind of phrase can follow “I am a”. Cross-attention can give high weight to the encoder representation corresponding to “étudiant”. The resulting source-conditioned decoder representation produces a high probability for “student”.

$I \text{ am } a \rightarrow \text{student}$

I. Generate EOS

I am a student $\rightarrow \text{EOS}$

EOS tells the decoder to stop. Final translation:

I am a student

23. One decoder step mathematically

Suppose the current target prefix is “I am a”. Let D be its target-side representation and H the encoder memory.

$Q_{\text{dec}} = D W_Q$

$K_{\text{enc}} = H W_K$

$V_{\text{enc}} = H W_V$

$A_{\text{cross}} = \text{softmax}(Q_{\text{dec}} K_{\text{enc}}^T / \sqrt{d_k})$

$$C = A_{\text{cross}} V_{\text{enc}}$$

C is source-conditioned information. It is combined through the decoder's residual paths and processed by the FFN. Finally:

$$\text{logits} = h_{\text{final}} W_{\text{vocab}} + b$$

The token with the selected probability becomes the next output token.

24. Training — how the model learns

During training the correct target sentence is known. Teacher forcing feeds the correct previous target tokens to the decoder.

Decoder input: I am a student

Target labels: I am a student EOS

Causal masking still prevents each position from seeing future target labels. The model predicts all next-token positions during a training pass.

25. Loss and backpropagation

For a correct token y with predicted probability $p(y)$, the token cross-entropy loss is:

$$\text{Loss} = -\log p(y)$$

High probability for the correct token gives low loss. Low probability gives high loss. The total loss drives backpropagation, which updates the learned parameters throughout the network: embeddings, Q/K/V projections, output projections, FFN weights, normalization parameters and other learned components.

26. Training vs inference

Training	Inference
Correct target prefix available	Only generated prefix available
Teacher forcing	Autoregressive generation
Causal mask allows parallel target-position computation	Tokens are generated sequentially
Loss compared with known labels	No target labels available

27. Why both attention types are needed in the decoder

Component	Main question
Masked self-attention	What target tokens have I generated, and how do they relate?
Cross-attention	Which source information is relevant right now?
FFN	How should I transform the information gathered at this position?

28. The three attention mechanisms

Type	Q	K	V	Purpose
Encoder self-attention	Encoder	Encoder	Encoder	Build source context
Decoder masked self-attention	Decoder	Decoder	Decoder	Use previous target context without future leakage
Cross-attention	Decoder	Encoder	Encoder	Read source information while generating target

29. Why attention is powerful

Standard self-attention provides direct token-to-token interaction, supports highly parallel matrix computation during training, and allows content-dependent information routing. Multi-head attention provides multiple learned representation subspaces.

The main cost is that the dense attention matrix is $n \times n$ for sequence length n , giving the standard mechanism approximately quadratic dependence on sequence length.

30. Original Transformer vs modern LLMs

Architecture	Example/family	Typical role
Encoder-only	BERT-style	Understanding/representation
Decoder-only	GPT-style	Autoregressive generation
Encoder-decoder	Original Transformer, T5-style	Sequence-to-sequence generation

Modern decoder-only LLMs generally do not have a separate encoder and therefore do not use encoder-decoder cross-attention. They use causal self-attention over the growing context.

31. Common mistakes to avoid

1. Encoder output is not a translated sentence — it is contextual source memory.
2. Cross-attention is the bridge: Q comes from decoder, K/V from encoder.
3. FFN does not mix tokens; attention does.
4. LayerNorm is not BatchNorm.
5. Softmax does not “translate”; it converts logits to probabilities.
6. The model does not contain a simple hard-coded word dictionary; translation behavior is learned across the network.

32. Final architecture — memorize this

SOURCE TEXT

↓ tokenize

TOKEN IDS

↓ embedding + position

ENCODER

■ ■ Multi-Head Self-Attention

■ ■ Residual + LayerNorm

■ ■ Feed-Forward

■ ■ Residual + LayerNorm

↓ repeat ×6

ENCODER MEMORY H

↓ K, V

DECODER TARGET PREFIX

■ ■ Masked Multi-Head Self-Attention

■ ■ Residual + LayerNorm

■ ■ Cross-Attention Q=decoder, K/V=encoder

■ ■ Residual + LayerNorm

■ ■ Feed-Forward

■■ Residual + LayerNorm
 ↓ repeat ×6
 LINEAR → VOCABULARY LOGITS → SOFTMAX → NEXT TOKEN
 ↓
 append token and repeat until EOS

33. The deepest mental model

Encoder: “Let every source token communicate with other source tokens, then transform the resulting representations.”

Decoder self-attention: “Let the target tokens generated so far communicate, but never reveal the future.”

Cross-attention: “Use the current target-side question to retrieve relevant information from the source memory.”

FFN: “Transform the information at each position.”

Linear + Softmax: “Turn the final representation into a probability distribution over the next token.”

Autoregressive loop: “Choose a token, append it, and run the decoder again until EOS.”

34. Final one-minute explanation

A Transformer converts tokens into embeddings and adds positional information. The encoder repeatedly uses self-attention so source tokens can communicate, then applies residual connections, normalization and feed-forward networks. The final encoder output is a contextual memory containing a vector for each source position.

The decoder starts with BOS and generates one target token at a time. Its masked self-attention sees only previous target tokens. Its cross-attention uses decoder Queries against encoder Keys and Values, allowing the decoder to retrieve source information relevant to its current generation step. After further normalization and feed-forward processing, a linear vocabulary projection produces logits and softmax turns them into next-token probabilities.

The selected token becomes part of the next decoder input. This repeats until EOS. The ability to translate is learned during training by minimizing next-token prediction loss across many source-target examples.

35. Final checklist

- What are embeddings and why are they needed?
- Why does the Transformer need positional information?
- What do Query, Key and Value mean?
- Why is QK^T computed?
- Why divide by $\sqrt{d_k}$?
- Why use softmax?
- Why multiply attention weights by V?
- What does multi-head attention add?
- What is a residual connection?
- What is LayerNorm?
- How is LayerNorm different from BatchNorm?
- What exactly does the FFN do?
- Why is decoder self-attention masked?
- In cross-attention, where do Q, K and V come from?
- How does encoder information reach the decoder?

- Where are target vocabulary probabilities produced?
- How does the model generate a complete translation?
- How does training differ from inference?